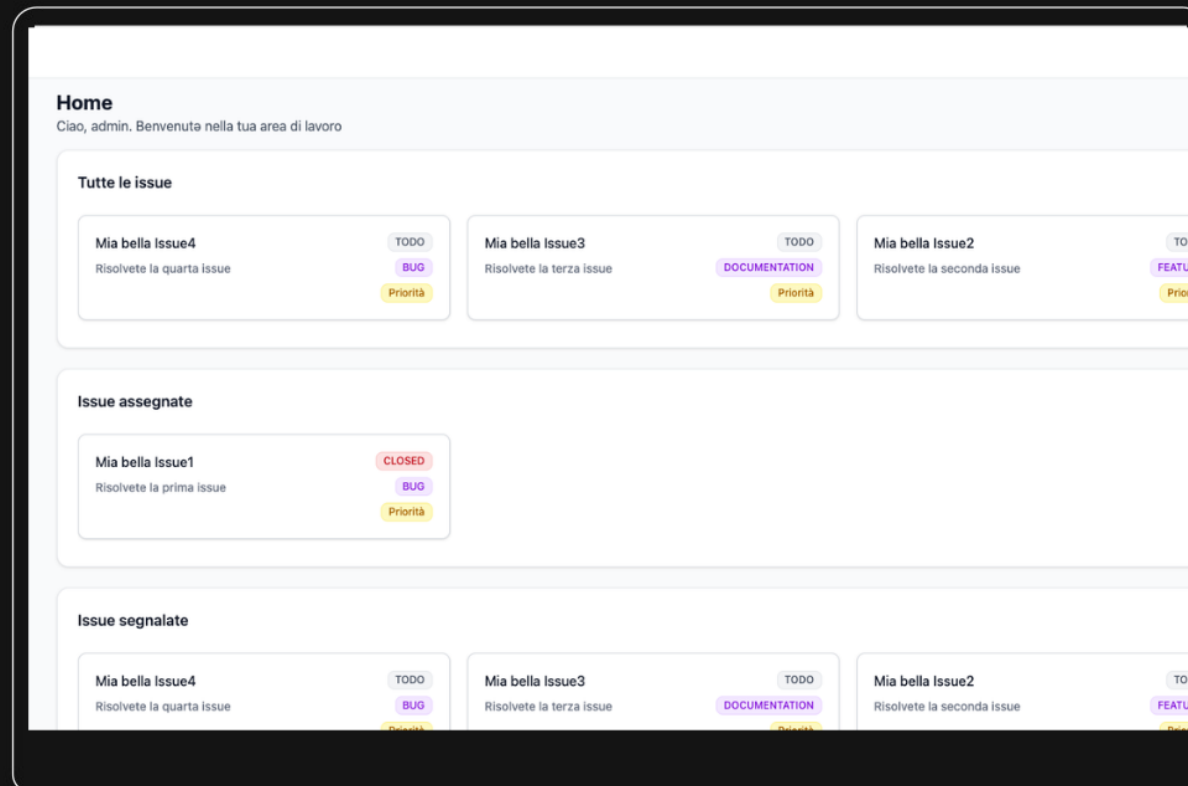


BugBoard2026

Petraccone S., Picari A., Sorrentino A.

Ingegneria del Software | A.A. 2025/2026



Introduzione e Obiettivi

BugBoard2026 è una piattaforma web-based per la gestione completa delle Issue nei progetti software.

Offre un approccio che favorisce la collaborazione tra i membri di un team; segnalare un bug e assegnarlo a un collega, rendendo il processo di gestione e risoluzione delle Issue, fluido e automatico.



Chi lo Usa

Progettato per sviluppatori, tester e stakeholder con ruoli e responsabilità ben definiti.



Funzionalità Core

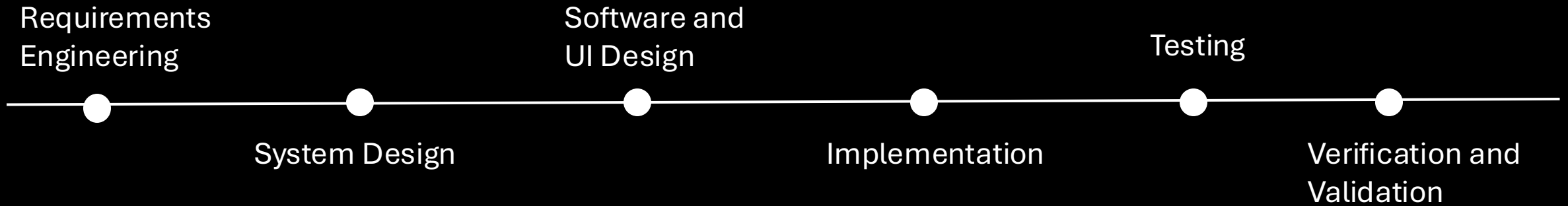
Segnalazione, assegnamento e tracciamento delle Issue.



Valore Principale

Centralizzare la gestione dei bug, migliorando la collaborazione tra team e la qualità del software.

Modello di sviluppo Waterfall



Attori e Casi d'Uso

Il sistema prevede tre categorie di utenti, ciascuna con ruoli e permessi distinti



Admin

Utente del sistema con *privilegi elevati*

- Gestione utenze
- Assegnazione Issue
- Modifica Issue
- Supervisione sistema



User

Utente del sistema con *privilegi standard*

- Segnalazione Issue
- Allegare immagini/tag..
- Risoluzione Issue assegnate
- Aggiornamento dello stato



Lurker

Utente del sistema con *privilegi ridotti*

- Visualizzazione Issue
- Accesso sola lettura
- Stakeholder esterni

Funzionalità



Sistema di Filtraggio e Ordinamento

- per tipo, stato e priorità
- paginazione per gestire molte Issue in modo efficiente



Esportazione dei dati in formato CSV



Cronologia delle modifiche delle Issue



Notifiche di risoluzione Issue

Ciclo di vita di una Issue

Creazione Issue

Si creano nuove segnalazioni andando a definire: titolo, descrizione, tipo e priorità.

Assegnazione Guidata

Assegnazione di un qualsiasi bug a un membro del team, ricevendo una notifica automatica.

Risoluzione e/o Chiusura

Requisiti Non-Funzionali



Interfacce Utente

Interfaccia responsive su dispositivi mobili e desktop, adattandosi dinamicamente alle diverse risoluzioni.



Performance

Le operazioni comuni (caricamento lista issue, login) si completano in tempi rapidi e accettabili per l'utente.



Sicurezza

Accesso protetto tramite autenticazione e comunicazioni cifrate tra client e server via HTTPS.



Scalabilità

Architettura distribuita per scalare indipendentemente Frontend e Backend sotto carichi crescenti.

Architettura e Deployment

Il sistema è progettato su un'architettura cloud moderna, con servizi separati e indipendenti per garantire scalabilità, resilienza e aggiornamenti automatizzati.

Gestione dai webhook di GitHub



Frontend

L'interfaccia utente in *React* è distribuita tramite *Vercel*
- Single Page Application.



Backend

API REST e la logica di business gestite da *Spring Boot* sono ospitate su *Render* come web service.



Database

I dati persistenti sono salvati su un'istanza *PostgreSQL* in cloud, connessa in modo sicuro al backend.

Separation of Concerns (SoC)

Dividere le responsabilità per
garantire l'indipendenza

Frontend

Backend

Vantaggi?

Gestione autonoma dei dati e della
logica

Possibilità di aggiornare il client
senza impattare il server.

Flessibilità tecnologica

Layered architecture

Controller

Gestisce l'ingresso delle richieste

Service

Contiene la logica di business e
coordina le operazioni

Repository

Isola il database dal resto
dell'applicazione

Vantaggi?

Progettare a livelli permette di favorire
l'astrazione del codice. Le modifiche al
database rimangono isolate nel Repository.

Inversion of Control (IoC) e Dependency injection

Il design sfrutta l'iniezione delle
dipendenze per eliminare i vincoli rigidi
tra le classi del sistema.

È possibile sostituire
l'implementazione di un componente
senza alterare chi ne fa uso,
facilitando la manutenzione.

Design Pattern Applicati

Dependency Injection

Garantisce basso accoppiamento delle classi rendendo i moduli altamente testabili.

Repository Pattern

Garantisce l'isolamento della logica di persistenza dal resto dell'applicazione, permettendo al livello Service di manipolare i dati vedendoli come oggetti Java.

Data Transfer Object (DTO)

Utilizzate classi DTO intermedie per modellare i payload delle richieste e delle risposte. Questa scelta protegge dati sensibili ed evita il sovraccarico di informazioni restituite nel JSON di risposta.

Model View Controller (MVC)

utilizzato per separare le responsabilità dei ruoli all'interno del Backend.

- MODEL (Entità e DTO)
- VIEW (il payload)
- CONTROLLER (intercettare le richieste HTTP e delegare l'esecuzione della relativa logica di business ai moduli Service.)

Pattern Builder

L'utilizzo di questo pattern ha permesso di instanziare oggetti complessi e DTO in modo fluido, leggibile e immutabile.

Comunicazioni

Ciclo di vita di una Richiesta

1 Navigazione

accede tramite browser all'applicazione React, servita dalla **CDN** globale di Vercel.

Controllo sicurezza 2

richiesta **HTTP OPTIONS** verso il backend su Render. Il filtro **CORS** di **Spring Security** intercetta la chiamata e valida l'origine(Vercel), autorizzando la transazione.

3 Chiamata API autenticata

Il frontend invia la richiesta **REST** effettiva **allegando il token JWT** nell'header di autorizzazione. Spring Boot elabora la logica di business.

Persistenza dati 4

Il server Spring Boot comunica con l'istanza cloud del database PostgreSQL tramite **connessione JDBC**. restituendo infine la risposta in formato JSON al client.

Qualità del codice

Utilizzando **SonarQube**

Security e Reliability

0 bug
0 vulnerabilità
0 Security Hotspots

Maintainability

Il sistema mantiene un Rating A.
Livello di Technical Debt è irrilevante e i pochi code smell residui

Duplication

Il codice presenta un perfetto 0.0% di codice duplicato su oltre 3.200 righe analizzate, evidenziando l'uso rigoroso del riuso dei componenti e della modularità.

Test Coverage

La percentuale si attesta attorno al 21%
I test sono stati concentrati esclusivamente sui metodi non banali e critici della logica di business , aventi almeno due parametri.

Testing